Übungsblatt 8: I/O und Systemarchitektur

Mikrocomputertechnik 1

Musterlösung

Ziele und Vorgehen

Sie unterscheiden Adresse, Maske und Datenwort, erkennen Pegel- und Ereignisverträge und erklären Trap-Eintritt, Kontextrettung und Dienstanfragen. Teil A ist ohne Simulator lösbar und übt Klausuraufgaben. Teil B vergleicht Vorhersagen mit Ripes und Venus.

Orientierung: Papieraufgaben etwa 70 Minuten, 60 Punkte zur Selbsteinschätzung; praktische Aufgaben etwa 30 Minuten. Für eine gemeinsame kürzere Papierphase eignen sich Aufgaben 1, 3 und 5. Begründen Sie Entscheidungen und nennen Sie bei Rechnungen Größen und Einheiten.

Gemeinsame Annahmen

- RV32, 32-Bit-Register, Vier-Byte-Instruktionen in den Beispielen. Little Endian, gültige wortausgerichtete Speicherzugriffe.
- MMIO (Memory-Mapped I/O): Geräteregister über Adressen. Papierkarte: Schalterbasis `0xF0000000`, mindestens vier aktiv-High-Eingänge, Schalter n an Bit n; Pixelbasis `0xF0000020`, acht Pixel mit vier Byte Abstand. Format `0x00RRGGBB`. Diese Adressen sind Lehrannahmen, keine universellen Ripes-Konstanten.
- Pegelaufgaben: Eingang null oder eins ohne Lese-Nebenwirkung. Druck-Freigabe-Erkennung startet bei freigegeben. Der Dreiprobenfilter startet mit stabilem Zustand null und übernimmt eine Änderung erst nach drei gleichen aufeinanderfolgenden Proben.
- Privilegiertes Modell: Machine Mode M und User Mode U vorhanden, keine Delegation, keine verschachtelten Traps. Soweit nicht anders angegeben, mtvec im Direct-Modus.
- Statusfelder im RV32-Grundmodell: MIE in Bit 3, MPIE in Bit 7, MPP in Bits 12:11; U wird als 0, M als 3 codiert. Der aktuelle Modus ist von MPP zu unterscheiden.
- CSRs (Control and Status Registers) enthalten Ursache, Fortsetzungsadresse und Status. Keine automatische Sicherung aller Integerregister. Handlerstack gültig, sp auf 16 Byte ausgerichtet.
- Freigabeaufgaben: Programm läuft gerade in M, betrachtete Quelle nach M gerichtet, weitere Prioritäten geklärt. P ist anstehende Meldung, E die Quellenfreigabe, MIE globale Freigabe in M.
- Venus-Kursvertrag: Kennung in a0, erstes Argument in a1. Dienste 1 Ganzzahl, 4 nullterminierter Text, 10 Beenden. Daraus folgt keine allgemeine Registererhaltung.
- Die privilegierten Trap-Rechnungen sind Papiermodelle; Konsolenausgabe und MMIO-Versuch beweisen keinen ausgeführten M-Mode-Handler.

Teil A: Papieraufgaben

Aufgabe 1: Adresse, Auswahl und Farbwort (12 Punkte)

1. Füllen Sie die Tabelle für die Maske acht und das Schalter-Pixel-Echo aus. (4 P.)

Rohwort	Maskenergebnis	Gewähltes Farbwort
`0b0000`		
`0b0101`		
`0b1000`		
`0b1101`		

2. Bestimmen Sie die Adressen der Pixel 3 und 7 sowie die erste Adresse unmittelbar hinter dem Matrixbereich. Darf dorthin für Pixel 8 geschrieben werden? (3 P.)
3. Das rote Wort `0x00FF0000` wird mit einem Vier-Byte-Store geschrieben. Geben Sie die Bytes an aufsteigenden Adressen an. Ist das Maskenergebnis acht schon die Farbe Rot? (2 P.)
4. Leiten Sie die Boolesche Auswahl S_F für eine oberste Hexziffer F aus A_{31} bis A_{28} her. Bestimmen Sie S_F für `0xF0000000` und `0xE0000000`. Wählt S_F allein schon das Schalterregister? (3 P.)

Musterlösung Aufgabe 1

Rohwort	Maskenergebnis	Farbwort
`0b0000`	0	`0x00000000`
`0b0101`	0	`0x00000000`
`0b1000`	8	`0x00FF0000`
`0b1101`	8	`0x00FF0000`

2. $A_3 = 0xF0000020 + 12 = 0xF000002C$, $A_7 = 0xF0000020 + 28 = 0xF000003C$. Die exklusive Grenze ist `0xF0000040`. Pixel 8 ist außerhalb der acht Zellen.
3. Little Endian: `00 00 FF 00` an aufsteigenden Adressen. Die Maske acht isoliert ein Eingangsbit; erst die Entscheidung bildet Rot.
4. $S_F = A_{31} \wedge A_{30} \wedge A_{29} \wedge A_{28}$. Für F (1111) ist es eins, für E (1110) null. Das erkennt nur den Bereich; weitere Bits und Zugriffssignale wählen das Register.

Aufgabe 2: Polling und Entprellung (12 Punkte)

1. Eine Abfrage kostet bei angenommenen 10 MHz hundert Takte und erfolgt alle fünf Millisekunden. Berechnen Sie ihre Dauer und den Prozessorzeitanteil. Welche zusätzliche Wartezeit bis zur nächsten Prüfung kann bei einem bestehen bleibenden Pegel entstehen? (4 P.)
2. Ein reines Pegelregister wird bei 0, 5 und 10 ms gelesen. Ein Puls liegt vollständig zwischen 6 und 8 ms. Wird er erkannt? Nennen Sie eine passende Gegenmaßnahme. (2 P.)
3. Betrachten Sie die Rohfolge unten. Wie viele Ereignisse zählt bloßes Warten auf Druck und Freigabe? Tragen Sie außerdem den Zustand des vereinbarten Dreiprobenfilters nach jeder Probe ein und bestimmen Sie seine Druckereigniszahl. (4 P.)

Probe	1	2	3	4	5	6	7	8	9	10	11
Rohwert	0	1	0	1	1	1	0	1	0	0	0
Stabil											

4. Ist eine Schleife mit 10000 Iterationen ein Nachweis für 20 ms Entprellzeit? Welche Informationen fehlen? (2 P.)

Musterlösung Aufgabe 2

1. $100/10^7\,\mathrm{s} = 10\,\mathrm{s}$. $10\,\mathrm{s}/5000\,\mathrm{s} = 0{,}002 = 0{,}2\,\%$. Bis zur Prüfung kann ungefähr eine Periode von fünf Millisekunden vergehen; anschließend kommt die Verarbeitung hinzu.
2. Nein, beide benachbarten Abfragen lesen null. Möglich sind eine ausreichend schnelle Abfrage, dokumentierte Ereignisspeicherung oder ein geeigneter Interruptpfad. Eine beliebige Ripes-Version wird dadurch nicht als IRQ-fähig vorausgesetzt.
3. Die Druck-Freigabe-Erkennung zählt an den Proben 2, 4 und 8: drei Ereignisse. Filterzustände: `0 0 0 0 0 1 1 1 1 1 0`. Bei Probe 6 wird eins akzeptiert, bei Probe 11 null; genau ein Druckereignis. Das Modell filtert diskrete Proben, beweist keinen durchgehend stabilen Analogpegel.
4. Nein. Mindestens Taktfrequenz, tatsächliche Maschinenbefehle und Taktkosten sowie weitere Verzögerungen fehlen. Anzahl der Quellzeilen oder Iterationen allein legt keine reale Zeit fest.

Aufgabe 3: Trap-Ursache, Ziel und Fortsetzung (12 Punkte)

Die folgenden Fälle sind unabhängig. Alle angegebenen mtvec-Werte werden von der betrachteten Implementierung unterstützt.

1. Decodieren Sie Art und Bedeutung von `0x00000008`, `0x0000000B` und `0x8000000B` in mcause. (3 P.)
2. Bestimmen Sie die Zieladresse für alle vier Fälle. Beschreiben Sie die Rolle der beiden niedrigsten mtvec-Bits. (5 P.)

mtvec	mcause	Handlerziel
`0x00000100`	`0x8000000B`	
`0x00000101`	`0x8000000B`	
`0x00000101`	`0x00000008`	
`0x00000101`	`0x0000000B`	

3. mepc ist in drei unabhängigen Fällen `0x00400040`. Bestimmen Sie eine begründete Fortsetzungsadresse für einen Interrupt, einen bewusst vollständig bearbeiteten 32-Bit-ecall und einen behobenen Fehler, dessen Instruktion erneut versucht werden soll. Warum genügt nicht immer `mepc += 4`? (4 P.)

Musterlösung Aufgabe 3

1. `0x00000008`: Exception, Code 8, Environment Call aus U. `0x0000000B`: Exception, Code 11, Environment Call aus M. `0x8000000B`: Interrupt, Code 11, Machine External Interrupt. Erst Interruptbit und Code gemeinsam bestimmen die Ursache.
2. Die Zieladressen in Reihenfolge sind `0x100`, `0x12C`, `0x100` und `0x100`. MODE 00 ist Direct; MODE 01 ist Vectored. BASE ist jeweils `0x100`. Nur Interrupts im Vectored-Modus addieren $4 \cdot 11 = 44$ Byte.
3. Interrupt: `0x00400040`, gespeicherte Fortsetzung. Bearbeiteter ecall: `0x00400044`, damit die Anfrage nicht erneut ausgeführt wird. Erneut zu versuchender Fehler: `0x00400040`. Eine pauschale Erhöhung könnte richtige Arbeit überspringen oder eine gewollte Wiederholung verhindern.

Aufgabe 4: Freigabebits und Statusänderungen (10 Punkte)

1. In einem unabhängigen Bitbeispiel ist mstatus `0x00000080`; alle anderen hier nicht behandelten Felder seien null. Welches Bit setzt die folgende Sequenz, welcher Wert entsteht und welches relevante Bit ginge beim Ersetzen des ganzen Registers durch acht verloren? (3 P.)

```
li    t1, 8
csrrs x0, mstatus, t1
```

2. Tragen Sie für das vereinbarte laufende M-Programm die Annahmeentscheidung ein. (3 P.)

Pending P	Quellenfreigabe E	MIE	Interrupt angenommen?
1	0	1	
1	1	0	
1	1	1	

3. Verhindert MIE gleich null einen ecall oder eine Illegal-Instruction-Exception? Begründen Sie. (1 P.)
4. Ein anderer Fall startet im aktuellen Modus M mit MIE gleich eins, MPIE gleich null und MPP gleich U. Ein Interrupt wird angenommen, der Handler aktiviert keine weiteren Interrupts und kehrt mit mret zurück. Ergänzen Sie die Zustände. (3 P.)

Feld	Vor Eintritt	Im Handler	Nach mret
Aktueller Modus	M		
MIE	1		
MPIE	0		
MPP	U		

Musterlösung Aufgabe 4

1. Maske acht setzt Bit 3, MIE. Es entsteht `0x00000088`. Beim ganzen Schreibversuch mit acht ginge das vorher gesetzte MPIE in Bit 7 verloren. csrrs erhält die übrigen schreibbaren Bits; x0 verwirft den gelesenen alten Wert.
2. Nein, nein, ja. Es gilt im vereinbarten M-Fall $P \wedge E \wedge$ MIE.
3. Nein. Diese synchronen Exceptions werden nicht durch die maskierbare M-Interrupt-Freigabe verhindert.

Feld	Vor Eintritt	Im Handler	Nach mret
Aktueller Modus	M	M	M
MIE	1	0	1
MPIE	0	1	1
MPP	U	M	U

MPP nach mret bezeichnet nicht den dann aktuellen Modus. Die Rückkehr verwendete den zuvor gespeicherten Modus M; danach wird MPP im angenommenen M/U-System auf U zurückgesetzt.

Aufgabe 5: Unterbrochenen Registerwert erhalten (8 Punkte)

Anfang: gültiger ausgerichteter Handlerstack, `sp = 0x2000`, `t0 = 7`. Keine verschachtelten Traps. Das folgende Lehrfragment verändert ausschließlich t0 und sp und enthält keinen Unteraufruf.

```
addi sp, sp, -16
sw   t0, 12(sp)
li   t0, 99
lw   t0, 12(sp)
addi sp, sp, 16
mret
```

1. Bestimmen Sie den reservierten Framebereich, neuen sp und die Sicherungsadresse von t0. (3 P.)
2. Bestimmen Sie t0 und sp nach dem Epilog. Warum muss ein Interrupt-Handler auch ein normalerweise caller-saved t-Register erhalten? (3 P.)
3. Darf zwischen Sicherung und Wiederherstellung ein beliebiges ABI-konformes `call` ergänzt werden? Nennen Sie zusätzliche gefährdete Register und die Größe eines Frames allein für `ra`, `t0`–`t6` und `a0`–`a7`. (2 P.)

Musterlösung Aufgabe 5

1. Frame `[0x1FF0, 0x2000)`, 16 Byte. Neuer sp `0x1FF0`. Die Sicherungsadresse ist $0x1FF0 + 12 = 0x1FFC$.
2. t0 ist nach dem Laden wieder 7, sp wieder `0x2000`. Der asynchron unterbrochene Code hat keinen normalen Aufruf vorbereitet und konnte einen benötigten t-Wert nicht selbst vorher sichern. Hardware-Trap-CSRs enthalten keine Kopie von t0.
3. Nein. Ein call verändert ra und kann sieben t- und acht a-Register verändern. Zusammen sind das 16 Wörter, also 64 Byte; zusätzlich selbst veränderte s-Register sind zu berücksichtigen. Der 16-Byte-Leaf-Frame genügt nicht.

Aufgabe 6: Dienst- und Funktionsvertrag trennen (6 Punkte)

1. Vor Venus Print Integer stehen `a0 = 4` und `a1 = 42`. Welche Anfrage erfolgt tatsächlich? Wie lauten die Werte für die beabsichtigte Ganzzahlausgabe? Ist die Dienstkennung eine ISA-Konstante? (2 P.)
2. Warum ersetzt ein ecall keine gewöhnliche Funktion mit jal-Link nach ra? Kann aus dem Namen s0 allein eine Erhaltung über jeden ecall gefolgert werden? (2 P.)
3. In einer fiktiven, ausdrücklich vereinbarten Lehrumgebung darf der Dienst t0, a0 und a1 ändern, erhält aber s0, sp und den benutzten Speicher. Ein Zähler soll mehrere Anfragen überleben. Nennen und begründen Sie zwei zulässige Strategien unter genau diesem Vertrag. (2 P.)

Musterlösung Aufgabe 6

1. Kennung vier ist Print String; 42 wird als Adresse einer nullterminierten Bytefolge behandelt. Ohne Speichervertrag ist das Ausgabeergebnis nicht garantiert. Für Print Integer gilt a0 gleich eins und a1 gleich 42. Die Kennung ist ein Venus-Vertrag, keine RISC-V-ISA-Vorgabe.
2. jal schreibt einen normalen Rückkehrlink. ecall löst eine Environment-Anfrage beziehungsweise einen Trap aus; seine Fortsetzung und Registeränderungen folgen dem Umgebungsvertrag. s0 ist nicht allein aufgrund der Funktionsrolle über beliebige Dienste garantiert.
3. Zähler in s0 halten, weil seine Erhaltung ausdrücklich vereinbart ist. Oder t0 vor der Anfrage auf einem gültigen ausgerichteten Stack sichern und danach zurückladen, weil sp und der benutzte Speicher ausdrücklich erhalten bleiben. Das ist kein behaupteter Venus-Clobber-Vertrag.

Teil B: Werkzeuge

Aufgabe 7: Schalter und Pixel in Ripes

Dokumentieren Sie Ripes-Version und Prozessormodell. Legen Sie mindestens vier Schalter und eine LED-Matrix an. Lesen Sie Basisadresse, Größe, Zugriffbreite und exportierte Symbole aus der tatsächlichen Registerkarte ab. Die Zahlen aus Teil A dürfen nur übernommen werden, wenn diese Karte sie bestätigt.

1. Öffnen Sie `beispiele/vl8_sw3_rot.s`. Übernehmen Sie die passenden Exports für `SWITCHES_0_BASE` und `LED_MATRIX_0_BASE`.
2. Prüfen Sie die Schalterwerte 0, 5, 8 und 13. Protokollieren Sie Rohwert, Maske, effektive Store-Adresse und Farbwort sowie die sichtbare Anzeige.
3. Schalten Sie SW3 nach dem Rot-Zustand wieder aus. Welche Zeile sorgt dafür, dass die Anzeige tatsächlich schwarz wird?
4. Optional: Konfigurieren Sie genau eine Zeile mit acht Pixeln und verwenden Sie `beispiele/vl8_lauflich.s`. Prüfen Sie Offsets 0 bis 28, Löschen des alten Pixels und Rückkehr zum Anfang. Geben Sie die Delay-Schleife nicht als gemessene Millisekundenzeit aus.

Eine Geräteansicht oder Kabelfarbe allein belegt keinen vollständigen Zugriff. Prüfen Sie Programmwert, Adresse und Gerätewirkung gemeinsam. Die Aufgabe verlangt keinen Interruptpfad und keinen M-Mode-Handler.

Musterlösung Aufgabe 7: hergeleitete Erwartungen

Rohwerte 0 und 5 liefern Maskenergebnis null und Farbwort null, also Schwarz. 8 und 13 liefern acht und `0x00FF0000`, also Rot. Der Store trifft Pixel 0 an der tatsächlichen Matrixbasis. Im off-Zweig setzt `li t2, 0` das inaktive Farbwort, bevor beide Zweige denselben Store erreichen.

Das Lauflicht beginnt mit Löschen aller acht Zellen. Es setzt einen Pixel grün, löscht ihn nach der Verzögerung, erhöht den Offset um vier und beginnt nach 28 erneut bei null. Offset 32 ist eine Grenze, kein neunter Pixel. Ein konkreter Ripes-Lauf und bestimmte Exportadressen werden hier nicht behauptet.

Aufgabe 8: Ausgabe in Venus

Verwenden Sie die Venus-Kurskonvention und dokumentieren Sie die Variante.

1. Führen Sie `beispiele/vl8_hello.s` und `beispiele/vl8_eva.s` aus. Notieren Sie vor jeder Anfrage a0 und a1 und unterscheiden Sie Zahlenwert und Adresse.
2. Erklären Sie la gegenüber lw und die Rolle des Nullterminators.
3. Ergänzen Sie optional den Text `Ergebnis:` vor der Ganzzahlausgabe. Bereiten Sie die Rechenoperanden nach dem Textdienst erneut aus dem Datenwort vor, statt eine undokumentierte Erhaltung eines t-Registers anzunehmen.
4. Beenden Sie mit Dienst zehn. Warum braucht der Code keinen selbst geschriebenen mret-Handler?

Musterlösung Aufgabe 8: hergeleitete Erwartungen

Hello World gibt `Hello World!` mit Zeilenumbruch aus und beendet. Der erste Dienst hat a0 gleich vier, a1 die Adresse von msg. Exit hat a0 gleich zehn.

EVA liest das Wort 21, verdoppelt es zu 42 und gibt `42` mit Zeilenumbruch aus. Beim Zahlendienst ist a0 eins, a1 der Wert 42; beim Textdienst ist a0 vier, a1 die Adresse von newline. la lädt eine Adresse, lw den dortigen Wortinhalt. Der String endet am Nullbyte.

Für einen zusätzlichen Präfix wird zuerst dessen Adresse an Print String übergeben. Danach wird die Adresse von n neu geladen und dessen Wort gelesen, verdoppelt und an Print Integer übergeben. Eine weiterhin benötigte t-Registerbelegung über den vorherigen Dienst wird so vermieden.

Venus behandelt diese Konsolenanfragen und die Fortsetzung intern. Die Ausgabe beweist keinen selbst programmierten privilegierten Trap-Handler. Ein tatsächlicher Venus-Lauf wird in dieser Musterlösung nicht protokolliert.